# 전자결재 (Approval) — 개발 자료

**대상 모듈**: `backend-core` 의 `approval` 도메인 + `ui` 의 `PageApproval.vue` **버전**: 2026-04-16 기준 실제 DB/코드 덤프 **용도**: 본 문서만 가지고 결재 기능을 확장 개발할 수 있는 실전 참조

---

## 0. 백엔드 구조 한눈에 (요약)

### 위치

```
backend-core/src/main/java/com/platform/v3/core/approval/
 ├─── ApprovalService.java             # DataSet 서비스 (공개 API)
 ├─── mapper/
 │    └─── ApprovalMapper.java          # MyBatis 인터페이스
 └─── flowable/                        # Flowable 엔진 리스너/델리게이트
      ├─── ApprovalAssigneeResolver.java
      ├─── ApprovalCompleteDelegate.java
      ├─── ApprovalNotificationListener.java
      └─── ApprovalProcessStartListener.java

backend-core/src/main/resources/
 ├─── mapper/approval/ApprovalMapper.xml      # SQL 143 lines
 └─── processes/                              # BPMN 4종 + DMN 내장
      ├─── sequential_approval.bpmn20.xml     # 순차 결재
      ├─── parallel_agreement.bpmn20.xml      # 병렬 합의
      ├─── dynamic_approval_line.bpmn20.xml   # 동적 결재선 (DMN 자동)
      └─── with_final_approval.bpmn20.xml     # 전결 포함
```

### DataSet API 엔드포인트 (모두 `POST /api/dataset/search` 로 호출)

| serviceName | 메서드 | 설명 | 응답 DataSet |
|---|---|---|---|
| approval/searchInbox | searchInbox | 결재함 조회 (boxType: PENDING/COMPLETED/DRAFTED..., userNo, keyword) | ds_inbox |
| approval/searchDetail | searchDetail | 문서 상세 + 결재선 | ds_doc, ds_line |
| approval/searchFormTemplates | searchFormTemplates | 양식 목록 (휴가/출장/구매/지출...) | ds_forms |
| approval/submitDocument | submitDocument | 문서 상신 + DMN 자동결재선 생성 + SSE 알림 | {docId, approvers} |
| approval/approve | approve | 해당 line 승인 → 전체 승인이면 문서 APPROVED, 아니면 IN_PROGRESS | {success, allApproved} |
| approval/reject | reject | 반려 처리 + 기안자 알림 | {success} |

### 핵심 로직

**상신 ( `submitDocument` ) 흐름**

1. `ds_doc` 에서 문서 row 추출 (drafterNo/drafterName 기본값: currentUser, status=PENDING)
2. `insertDocument` → `docId` 획득 (auto-increment)
3. `selectApproversForDocFromDmn(amount, formCode)` — DMN 규칙으로 결재선 자동 산출 (금액/양식 코드 기반)
4. 각 approver 를 `ap_approval_line` 에 stepOrder 순서대로 INSERT
5. `NotificationService.notify()` 로 첫 approver 에게 SSE 알림 발송
6. `log.info("결재 상신 완료: docId={}, approvers={}")`

**승인 ( `approve` ) 흐름**

1. 해당 `lineId` 의 status → APPROVED
2. 전체 결재선 재조회 → 모두 APPROVED 인지 검사
3. 모두 승인: 문서 status=APPROVED + 기안자에게 "전결 완료" SSE 알림
4. 일부만 승인: 문서 status=IN_PROGRESS

**반려 ( `reject` ) 흐름**

1. 해당 `lineId` status=REJECTED, comment 저장
2. 문서 status=REJECTED (한 명 반려로 문서 종결)
3. 기안자에게 "결재 반려 — {docTitle}" SSE 알림

### Mapper 메서드 ( `ApprovalMapper.java` , 37 lines)

- `selectInbox(boxType, userNo, keyword)` — 함 종류별 조회
- `selectDetail(docId)` / `selectApprovalLine(docId)`
- `selectFormTemplates()` — 양식 목록
- `insertDocument(row)` / `updateDocumentStatus(docId, status)`
- `insertApprovalLine(line)` / `updateApprovalLineStatus(lineId, status, comment)`
- `countPendingForUser(userNo)` — dashboard 위젯용
- `selectApproversForDocFromDmn(amount, formCode)` — **DMN 자동결재선**

### Flowable 통합 (선택적 경로)

`flowable/` 하위 4개 클래스는 BPMN 프로세스가 런타임에 트리거하는 Java 리스너:

- `ApprovalProcessStartListener` — 프로세스 시작 시 변수 초기화
- `ApprovalAssigneeResolver` — 각 결재 단계의 assignee 를 동적으로 결정 (DB lookup)
- `ApprovalCompleteDelegate` — 결재 완료 task 실행 (문서 status 업데이트)
- `ApprovalNotificationListener` — 프로세스 이벤트 → NotificationService SSE

**현재 `ApprovalService` 는 DB 기반 단순 결재선을 쓰고 있고, Flowable BPMN 은 고급 시나리오용으로 병렬 존재.** 두 경로 중 양식/조건에 따라 선택 가능한 구조.

### BFF (backend-bff) 경유 없음

결재는 **외부 서비스 연동이 없는 내부 도메인**이라서 `backend-bff` 가 거치지 않습니다. 포탈 UI → `axios.post('/api/dataset/search', { serviceName: 'approval/...', ... })` → `backend-core(19090)` 직접 호출 → 응답.

---

## 0.1 "이거만 알면 개발 가능?" — 커버리지 분석

**결론: 위 섹션 0 만으로는 6~7할 정도**. 실제 결재 앱(UI + 운영 기능)을 만들려면 아래가 추가로 필요합니다.

## 이미 파악된 것 (6할)

- DataSet API 6종 (inbox/detail/forms/submit/approve/reject)
- Mapper 메서드 10개 + 테이블 3종 존재 여부
- SSE 알림 연동 포인트
- Flowable BPMN 4종 존재 여부

## 추가로 파악해야 할 것 (4할) — 본 문서 후속 섹션에서 전부 해결

**1. DB 스키마 실체** → 본 문서 §2 에서 실측 덤프 제공

- `ap_document` / `ap_approval_line` 각 컬럼의 정확한 타입·제약·FK
- `amount` 단위 (원/천원/백만원?), `formCode` 코드체계
- 첨부 파일 테이블 ( `ap_attachment` ?) — **존재 여부 확인 완료: 없음, 신규 필요**
- 의견/회람/결재후 문서 연관 테이블

**2. DMN 자동결재선 규칙** → §4 에서 실제 쿼리 공개

- `selectApproversForDocFromDmn(amount, formCode)` 가 실제로 어떤 기준으로 결재자를 뽑는지
- 순수 SQL lookup 인지, Flowable DMN 엔진 호출인지 — **SQL 단일 쿼리**
- 금액 구간별 결재선 정의 위치 — **MyBatis `<choose>` 3구간 하드코딩**

**3. 결재선 고급 기능** → §6 의 "실제 사용 가능한 기능 vs 스텁" 표에서 전부 분류

- 전결 / 대결 / 회수 / 합의·협조 / 후결 / 재상신

**4. 양식(Form) 엔진** → §2.5, §6 에서 상태 명시

- 양식 템플릿 저장 형식 — **현재 없음. doc.content TEXT 자유 입력**
- 사용자 입력 → ds_doc 매핑 방식
- 결재 후 문서 이력 보존 방식

**5. 권한·접근제어** → §10 "주의 사항" 에서 요주의 지점 명시

- 부서별 열람 가능 범위
- 본인 기안·결재 문서만 vs 부서 전체
- `ROLE_APPROVER` 외의 추가 권한 체계
- `SecurityConfig` 및 DataSet 호출 시점의 `currentUser` 매핑

**6. UI 측 구현 상태 ( `PageApproval.vue` )** → §6.2 에서 상세 체크리스트

- 문서 상세 뷰 완성도
- 상신 폼, 결재선 편집 UI
- 버튼 액션 4종 (승인/반려/전결/대결)

**7. 외부 연동** → 별도 로드맵 §7 에서 처리

- 결재 완료 시 메일 발송 (Stalwart BFF MailPort)
- 모바일 푸시, 전자서명

## 본 문서로 9~10할 달성

아래 5개 파일의 실제 내용을 분석하여 본 문서 §2~§10 에 전부 반영했습니다:

1. `backend-core/src/main/resources/db/migration/V1__baseline.sql` → §2 DB 스키마
2. `backend-core/src/main/resources/mapper/approval/ApprovalMapper.xml` → §3 API, §4 DMN
3. `backend-core/src/main/resources/processes/dynamic_approval_line.bpmn20.xml` → §5 Flowable
4. `backend-core/src/main/java/com/platform/v3/core/approval/flowable/*.java` → §5.2 4 delegates

5. `ui/src/pages/PageApproval.vue` → §6.2 UI 스텝 분류

---

# 1. 아키텍처 한눈에

```
┌──────────────────────┐   POST /api/dataset/search      ┌──────────────────────────────┐
│ PageApproval │ ──────────────────────────────────────────────────────────▶ │
DataSetController │
│ .vue (19173) │   Authorization: Bearer <JWT>    │  (/api/dataset/*) │
└──────────────────────┘                                 └──────────────┬───────────────┘
                                                                        │ ServiceRegistry
                                                                        ▼
                                                          ┌──────────────────────────┐
                                                          │ ApprovalService  │  ← @DataSetServiceMapping
                                                          └──────┬──────────┬────────┘
                                                MyBatis mapper      │          │  NotificationService
                                                                    ▼          ▼
                                                        ┌──────────────────────────┐
┌──────────────────────┐
                                                        │  ApprovalMapper  │  │  cm_notification │
                                                        │     (XML SQL)    │  │  + SSE emitter  │
                                                        └──────────┬──────────────┘
└──────────────────────┘
                                                                   ▼
                                                  ┌──────────────────────────────────┐
                                                  │ ap_document              │
                                                  │ ap_approval_line         │
                                                  │ cm_code (FORM_CODE 7건)  │
                                                  │ org_employee/position    │
                                                  └──────────────────────────────────┘
```

**통신 프로토콜**: v1 호환 DataSet 규약

- 요청: `{ serviceName, datasets/parameters }`
- 응답: `{ data: { ds_xxx: { rows: [...] } } }`
- 인증: `Authorization: Bearer <Keycloak JWT>` (backend-core SecurityConfig 에서 검증)

**BFF 경유 없음** — 결재는 외부 서비스 연동이 없는 순수 내부 도메인이므로 `backend-bff(19091)` 이 아닌 `backend-core(19090)` 에 직접 호출한다.

**Flowable 경로는 선택적**: 현재 서비스 구현은 **DB 기반 단순 결재선**( `ApprovalService` ) 을 사용하고 있으며, Flowable BPMN 4종 + JavaDelegate 4종은 **병렬 존재**한다. 고급 시나리오(전결, 분기, 멀티 인스턴스) 필요 시 프로세스 런타임으로 전환 가능.

---

# 2. 데이터베이스 스키마 (실측 덤프)

## 2.1 `platform_v3.ap_document`

```
CREATE TABLE platform_v3.ap_document (
    doc_id       BIGSERIAL PRIMARY KEY,
    doc_title    VARCHAR(256) NOT NULL,
```

```
    form_code    VARCHAR(32)  NOT NULL,     -- cm_code.group_cd = 'FORM_CODE'
    drafter_no   VARCHAR(32)  NOT NULL,     -- org_employee.employee_no
    drafter_name VARCHAR(64)  NOT NULL,
    drafter_dept VARCHAR(64),
    status       VARCHAR(16)  NOT NULL,     -- DRAFT/PENDING/IN_PROGRESS/APPROVED/REJECTED
    content      TEXT,                      -- HTML 본문 (또는 JSON)
    created_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_ap_document_drafter ON platform_v3.ap_document(drafter_no);
CREATE INDEX idx_ap_document_status  ON platform_v3.ap_document(status);
```

**status enum (문자열 제약, DB 제약은 없음):**

- `DRAFT` : 임시저장
- `PENDING` : 상신 직후, 첫 결재자 대기
- `IN_PROGRESS` : 1인 이상 승인 + 잔여 단계 남음
- `APPROVED` : 전결 완료
- `REJECTED` : 반려됨

## 2.2 `platform_v3.ap_approval_line`

```
CREATE TABLE platform_v3.ap_approval_line (
    line_id       BIGSERIAL PRIMARY KEY,
    doc_id        BIGINT      NOT NULL REFERENCES platform_v3.ap_document(doc_id) ON DELETE CASCADE,
    step_order    INTEGER     NOT NULL,     -- 1부터 순차
    approver_no   VARCHAR(32) NOT NULL,
    approver_name VARCHAR(64) NOT NULL,
    role          VARCHAR(32),              -- 결재자 직책 (position_name)
    status        VARCHAR(16) NOT NULL DEFAULT 'PENDING',  -- PENDING/APPROVED/REJECTED
    comment       TEXT,                     -- 승인/반려 의견
    acted_at      TIMESTAMPTZ               -- 처리 시각
);
CREATE INDEX idx_ap_line_doc ON platform_v3.ap_approval_line(doc_id);
```

## 2.3 `platform_v3.cm_code` (양식 코드)

`group_cd = 'FORM_CODE'` 로 조회한 7건 (V7 seed):

| code | code_name | sort_order |
|---|---|---|
| LEAVE | 휴가신청서 | 1 |
| EXPENSE | 지출결의서 | 2 |
| PURCHASE | 구매요청서 | 3 |
| BIZTRIP | 출장신청서 | 4 |
| CONTRACT | 계약검토서 | 5 |
| HR | 인사품의서 | 6 |

| | | |
|---|---|---|
| IT | IT자산신청 | 7 |

### 2.4 `platform_v3.org_employee` (결재자 lookup 용)

```
employee_id      BIGSERIAL PRIMARY KEY,
employee_no      VARCHAR(32) NOT NULL,     -- 사번 (approver_no와 매칭)
employee_name    VARCHAR(64) NOT NULL,
dept_id          BIGINT NOT NULL,
position_id      BIGINT NOT NULL,          -- org_position.position_id
email            VARCHAR(128),
phone            VARCHAR(32),
keycloak_user_id VARCHAR(64),               -- Keycloak sub UUID 매핑
hire_date        DATE,
status           VARCHAR(16) DEFAULT 'ACTIVE'
```

DMN 자동 결재선 조회는 `org_employee JOIN org_position` 구조로 `position_level` 기준 필터링.

### 2.5 누락 / 참고 사항

| 기능 | DB 상태 | 설명 |
|---|---|---|
| 첨부 파일 | **미존재** | `ap_attachment` 테이블 없음. MinIO presigned URL 체계와 연계한 신규 테이블 필요 |
| 의견/댓글 | **없음** (ap_approval_line.comment 만 존재) | 결재자별 1회 의견. 다회 댓글 체인은 미지원 |
| 이력/버전 | **없음** | updated_at 만 있음. 전자결재 감사 로그 요건 시 audit 테이블 신규 |
| 양식 본문 템플릿 | **없음** | 현재는 doc.content TEXT 에 자유 입력. 양식별 동적 필드 스키마 저장소 필요 |
| 전자서명 | **없음** | - |
| 회람/참조선 | **없음** | ap_approval_line 은 결재자만. 참조자(CC) 저장소 신규 |

---

## 3. 서비스 API (DataSet)

모든 메서드는 `ApprovalService.java:22` 에 위치. 호출: `POST /api/dataset/search` , body `{ "serviceName": "<name>", "parameters": {...} }` .

### 3.1 `approval/searchInbox` — 결재함 조회

**입력**: `ds_search.rows[0]` 에

- `boxType` (default `PENDING` ): DRAFT / MY_DOCS / PENDING / IN_PROGRESS / COMPLETED / REJECTED / RECEIVED / CC_BOX / DEPT_BOX
- `userNo` (default `E0032` ): 사번
- `keyword` (선택): 제목/기안자명 LIKE 검색

**출력**: `{ ds_inbox: { rows: [문서 N건, 최대 200] } }`

- 각 row: `doc_id, doc_title, form_code, drafter_no, drafter_name, drafter_dept, status, created_at, updated_at`

**boxType 별 SQL 분기** ( `ApprovalMapper.xml:11~62` ):

| boxType | 조건 |
|---|---|
| DRAFT | `status='DRAFT' AND drafter_no=userNo` |
| MY_DOCS | `drafter_no=userNo` |
| PENDING | `status='PENDING'` + 해당 사용자 결재선에 PENDING 존재 |
| IN_PROGRESS | `status='IN_PROGRESS'` + 기안자 또는 결재자 |
| COMPLETED | `status='APPROVED'` + 기안자 또는 결재자 |
| REJECTED | `status='REJECTED'` + 기안자 또는 결재자 |
| RECEIVED | 결재선에 포함된 모든 문서 |
| CC_BOX | **미구현** (`AND FALSE` — 항상 0건) — 참조함 테이블 부재 |
| DEPT_BOX | `drafter_dept IS NOT NULL` (부서 조건은 구체화 안됨) |

### 3.2 `approval/searchDetail` — 문서 상세 + 결재선

**입력**: `ds_search.rows[0].docId` **출력**:

- `ds_doc` — 문서 메타 1건 ( `content` 포함)
- `ds_line` — 결재선 N건 ( `step_order` 오름차순, comment/acted_at 포함)

에러: docId 없음 → `BusinessException.badRequest` , 문서 미존재 → `notFound("문서를 찾을 수 없습니다: {docId}")` .

### 3.3 `approval/searchFormTemplates` — 양식 목록

**입력**: 없음 **출력**: `ds_forms.rows` — `cm_code WHERE group_cd='FORM_CODE' AND use_yn='Y'` 정렬

- 각 row: `form_code, form_name, sort_order`

### 3.4 `approval/submitDocument` — 상신 (Transactional)

**입력**: `ds_doc.rows[0]` 에 최소한 `docTitle, formCode, amount(선택), content(선택)` 포함. `drafterNo/drafterName` 미입력 시 `currentUser` 자동 세팅.

**처리 순서**:

1. 문서 INSERT → `docId` 획득 ( `keyProperty="docId" keyColumn="doc_id"` )
2. `selectApproversForDocFromDmn(amount, formCode)` — DMN 규칙으로 결재자 N명 자동 선정
3. 각 approver 를 `ap_approval_line` 에 `step_order=1, 2, 3...` 순서로 INSERT
4. 첫 결재자에게 SSE 알림 ( `NotificationService.notify(docId, 10L, "APPROVAL", "WEB", ...)` )

**출력**: `{ docId, approvers: N }` (approvers = 자동 생성된 결재선 수)

**주의**: `NotificationService.notify` 의 두 번째 인자(userId) 가 **하드코딩 10L** → `currentUser` 기반 매핑으로 교체 필요 ( `ApprovalService.java:95` , `:127` 라인).

### 3.5 `approval/approve` — 승인 (Transactional)

**입력**: `ds_search.rows[0]` 에 `lineId, docId, comment` **처리**:

1. `updateApprovalLineStatus(lineId, 'APPROVED', comment)` + `acted_at = NOW()`
2. `selectApprovalLine(docId)` 로 전체 라인 조회
3. `allApproved` (모두 APPROVED) → 문서 status= `APPROVED` + 기안자 "전결 완료" SSE
4. 아니면 문서 status= `IN_PROGRESS`

**출력**: `{ success: true, allApproved: boolean }`

### 3.6 `approval/reject` — 반려 (Transactional)

**입력**: `ds_search.rows[0].lineId/docId/comment` **처리**: 라인 REJECTED + 문서 REJECTED + 기안자에게 "결재 반려 — {docTitle}" SSE (comment 본문) **출력**: `{ success: true }`

---

## 4. DMN 자동 결재선 — 실제 구현 ( `selectApproversForDocFromDmn` )

> ***중요**: "DMN" 이라는 이름이지만 현재 구현은 **MyBatis** `<choose>` **단일 SQL** 이다. 진짜 Flowable DMN 엔진은 사용하지 **않는다**.*

```
SELECT e.employee_no AS approver_no, e.employee_name AS approver_name,
       p.position_name, p.position_level
FROM platform_v3.org_employee e
INNER JOIN platform_v3.org_position p ON e.position_id = p.position_id
WHERE e.status = 'ACTIVE'
  AND p.position_level <=
      CASE
        WHEN amount >= 10000000 THEN 1   -- 천만원 이상: level 1 (임원) 까지
        WHEN amount >=  1000000 THEN 2   -- 백만원 이상: level 2 (부서장) 까지
        ELSE 3                            -- 그 외: level 3 (팀장) 까지
      END
ORDER BY p.position_level
LIMIT 3
```

**한계점**:

- `formCode` 파라미터는 넘겨받지만 **실제로는 쿼리에서 무시됨** (금액 기반 필터만 적용)
- 최대 3명으로 하드코딩
- `ORDER BY p.position_level` 은 같은 레벨 여러 명일 때 랜덤성 → 전사 결재 테스트 시 일관성 부족
- 부서 승계, 기안자 상급자, 협조/합의 등 고급 정책 **미지원**

**개선 방향** (개발 시 참고):

1. `approval_line_rules` 테이블 신설: `(rule_id, form_code, min_amount, max_amount, position_levels, step_seq)`
2. 혹은 Flowable DMN 엔진 도입: `backend-core/src/main/resources/dmn/approval_line_rules.dmn` 을 `DmnEngine` 에 등록

3. `selectApproversForDocFromDmn` 을 `ApprovalLineResolver` 인터페이스로 추상화 → 전략 패턴으로 교체 가능

---

## 5. Flowable BPMN 병렬 구현 (현재 미사용)

### 5.1 BPMN 파일 4종

| 파일 | 프로세스 정의 | 용도 |
|---|---|---|
| `sequential_approval.bpmn20.xml` | 순차 결재 | 단순 1→2→3 순서 |
| `parallel_agreement.bpmn20.xml` | 병렬 합의 | 동일 레벨 복수 결재자 병렬 승인 |
| `dynamic_approval_line.bpmn20.xml` | DMN 동적 결재선 | Multi-Instance SubProcess + DMN 규칙 테이블 |
| `with_final_approval.bpmn20.xml` | 전결 포함 | 특정 단계 전결 시 상위 생략 |

### 5.2 Java Delegate / Listener 4종 ( `com.platform.v3.core.approval.flowable` )

| 클래스 | 타입 | 트리거 | 역할 |
|---|---|---|---|
| `ApprovalProcessStartListener` | `ExecutionListener` | processInstance start | 기안자에게 "상신 완료" SSE 알림 |
| `ApprovalAssigneeResolver` | `JavaDelegate` | BusinessRuleTask 대체 ServiceTask | `amount/formCode` → `approverList` 변수 세팅 |
| `ApprovalNotificationListener` | `ExecutionListener` | 각 UserTask start | 현재 차례 결재자에게 SSE 알림 |
| `ApprovalCompleteDelegate` | `JavaDelegate` | endEvent | 문서 status=`APPROVED` + 기안자 "전결 완료" 알림 |

**활성화 방법** (향후 전환 시):

1. `ApprovalService.submitDocument` 을 아래로 교체

```
runtimeService.startProcessInstanceByKey("dynamicApprovalLine",
    Map.of("docId", docId, "formCode", formCode, "amount", amount,
           "drafterId", drafterEmpId, "docTitle", title));
```

2. `approve/reject` 는 `taskService.complete(taskId, Map.of("decision", "APPROVE"|"REJECT"))` 로 대체
3. PageApproval UI 의 승인 버튼이 호출하는 `lineId` 를 `flowable taskId` 로 교체
4. `ap_approval_line` 테이블은 Flowable 의 `act_ru_task` / `act_hi_taskinst` 와 병행하거나 view 로 대체

---

## 6. 실제 사용 가능한 기능 vs 스텁 — 체크리스트

### 6.1 백엔드 (ApprovalService + Mapper)

| 항목 | 상태 | 설명 / 개발 필요 작업 |
|---|---|---|

| | | |
|---|---|---|
| 결재함 조회 (7 종) | ✅ 사용 가능 | DRAFT/MY_DOCS/PENDING/IN_PROGRESS/COMPLETED/REJECTED/RECEIVED |
| 결재함 `CC_BOX` (참조함) | ❌ 스텁 | `AND FALSE` 하드코딩. 참조자 테이블 + 쿼리 신규 |
| 결재함 `DEPT_BOX` (부서함) | 🟡 부분 | `drafter_dept IS NOT NULL` 뿐. "내 부서" 범위 필터 미구현 |
| 문서 상세 조회 | ✅ 사용 가능 | `ds_doc` + `ds_line` 2 DataSet 반환 |
| 양식 목록 | ✅ 사용 가능 | 7종 시드 데이터 존재 |
| 양식별 동적 필드 스키마 | ❌ 미구현 | 템플릿 엔진 필요 (JSON schema 기반 권장) |
| 상신 (단순 순차) | ✅ 사용 가능 | 자동 결재선(최대 3명) 생성, 첫 결재자 알림 |
| 상신 시 결재선 수동 편집 | ❌ 미구현 | UI 에서 결재자 선택 → `ds_line` 으로 전송, Service 수정 필요 |
| 승인 (전결 완료 판정) | ✅ 사용 가능 | allApproved 체크 후 문서 APPROVED |
| 반려 | ✅ 사용 가능 | 1명 반려 시 즉시 문서 REJECTED |

| | | |
|---|---|---|
| **전결** (상위 단계 생략) | ❌ 미구현 | 고위 결재자가 승인하면 남은 단계 자동 SKIP |
| **대결** (부재자 대신 결재) | ❌ 미구현 | 대리인 지정 테이블 + approve 에 `on_behalf_of` 파라미터 |
| **회수** (기안자 취소) | ❌ 미구현 | `PENDING/IN_PROGRESS` 상태에서 DRAFT 복귀 API |
| **합의/협조** (병렬) | ❌ 미구현 | BPMN parallel_agreement 존재하지만 Service 연결 안됨 |
| **후결** (사후 보고) | ❌ 미구현 | 긴급 상신 → 먼저 완료 후 결재자 승인만 받는 패턴 |
| **재상신** (반려 후 재전송) | ❌ 미구현 | REJECTED 상태에서 수정 → PENDING 전이 API |
| 대기자 카운트 (`countPendingForUser`) | ✅ 사용가능 | Mapper 에만 존재, Service 에서 아직 노출 안됨 (dashboard 연결 필요) |
| DMN 자동 결재선 | 🟡 부분 | 단일 SQL. `formCode` 파라미터 무시됨. 규칙 DB화 또는 Flowable DMN 엔진 도입 필요 |
| Flowable BPMN 4종 | 🟡 부분 | 파일 + Java Delegate 존재하나 Service 와 미연결 |
| SSE 알림 연동 | ✅ 사용가능 | 상신/승인/반려 3곳에서 notify. **userId 하드코딩 10L 제거 필요** |
| 첨부 파일 | ❌ 미 | 테이블 없음. `ap_attachment` + MinIO presigned 연동 신규 |

| | | |
|---|---|---|
| | 구현 | |
| 의견 댓글 체인 | ❌ 미구현 | 라인별 단일 comment 만 존재 |
| 전자서명 / 도장 이미지 | ❌ 미구현 | - |
| 감사 로그 (audit) | ❌ 미구현 | 상태 변경 이력 별도 저장소 필요 |

### 6.2 프론트엔드 ( `PageApproval.vue` )

| 항목 | 상태 | 설명 / 개발 필요 작업 |
|---|---|---|
| 9종 박스 네비게이션 | ✅ 구현 | 좌측 사이드 리스트 렌더링 |
| 결재함 목록 DataTable | ✅ 구현 | `doc_id/doc_title/drafter/status/created_at` 5컬럼 |
| 행 클릭 → 상세 이동 | ❌ 스텁 | `onRowClick` 이 `console.log` 만 함. 라우팅 / 모달 미구현 |
| 문서 상세 뷰 | ❌ 미구현 | 별도 컴포넌트 없음. 상신 폼 공유 or 신규 페이지 |
| 상신 폼 (양식 선택) | ❌ 미구현 | 양식 조회 API 연결 안됨 |
| 상신 폼 (동적 필드) | ❌ 미구현 | 양식별 필드 스키마 기반 렌더링 |
| 결재선 프리뷰/편집 | ❌ 미구현 | 상신 전 결재자 확인 + 수동 추가/제거 |
| 첨부 업로드 | ❌ 미구현 | MinIO presigned URL 호출 UI |
| 결재 액션 버튼 (승인/반려) | ❌ 미구현 | 상세 뷰에 없음 |
| 의견 입력 다이얼로그 | ❌ 미구현 | 승인/반려 시 코멘트 입력 |
| 결재선 시각화 | ❌ 미구현 | step_order 기반 타임라인 / 스텝바 |

| | | |
|---|---|---|
| 기안자 직책/부서 표시 | ❌ 미구현 | drafter_dept 는 목록에 없음 |
| searchInbox userId 파라미터 버그 | ❌ 버그 | UI 가 userId: 1 을 넘기지만 Service 는 userNo 문자열을 기대 (default 'E0032') — 결과 불일치 |

---

## 7. 개발 로드맵 (제안 순서)

### Phase A — 기반 버그 수정 (½ day)

1. `userId → userNo` **매핑 정리**: keycloak JWT `preferred_username` 또는 `employee_no` 클레임으로 currentUser 추출. `PageApproval.vue:51` 의 `userId: 1` 을 `useAuthStore().user.employeeNo` 로 교체.
2. `NotificationService.notify(..., 10L, ...)` **하드코딩 제거**: `drafterNo` → `org_employee.employee_id` 로 변환하여 전달.
3. `CC_BOX` **기본 동작 결정**: 신규 `ap_approval_cc` 테이블 만들거나, 쿼리를 `AND TRUE LIMIT 0` 으로 변경 (빈 탭 표시만).

### Phase B — UI 상세 뷰 + 액션 (1 day)

4. `components/approval/ApprovalDetailDialog.vue` 신규: `searchDetail` 호출 + `ds_doc` / `ds_line` 렌더링 + **PrimeVue Timeline** 으로 결재선 시각화.
5. 승인/반려 버튼 + `InputTextarea` 코멘트 입력 → `approval/approve` , `approval/reject` 호출.
6. 행 클릭 → 다이얼로그 오픈 ( `onRowClick` 수정).

### Phase C — 상신 폼 (1~2 day)

7. `components/approval/ApprovalSubmitDialog.vue` 신규: 양식 `Dropdown` + 제목 입력 + `md-editor-v3` 본문 + 금액 `InputNumber` .
8. 제출 전 `selectApproversForDocFromDmn` 프리뷰 호출 → 결재선 미리보기.
9. `submitDocument` 호출 → 성공 시 결재함 재조회 ( `PENDING` 탭).
10. **선택**: 결재자 수동 추가 ( `PickList` 로 `org_employee` 검색).

### Phase D — 첨부 (½ day)

11. DB: `ap_attachment (attach_id, doc_id, object_key, filename, size, mime, uploaded_at)` 추가 마이그레이션 `V8__approval_attachments.sql` .
12. Mapper: `insertAttachment` , `listAttachments(docId)` .
13. Service: `approval/uploadAttachment` , `approval/listAttachments` DataSet 메서드.
14. UI: MinIO `/api/bff/storage/presigned?op=PUT` 호출 → S3 PUT → 업로드 후 Service 호출로 메타 저장.

### Phase E — 고급 기능 (선택)

15. **전결**: `ap_approval_line.is_final_approver` 플래그 + `approve` 에서 true 시 나머지 라인 SKIP.
16. **대결**: `ap_delegation(delegator_no, delegatee_no, from_date, to_date)` 테이블 + `approve` 시 대리 플래그 기록.
17. **회수**: `approval/withdraw` API — `status IN ('PENDING','IN_PROGRESS') AND drafter_no = currentUser` 체크 후 DRAFT 복귀.
18. **재상신**: `approval/resubmit` — REJECTED → 새 docId 복제 + 원본 참조 저장.
19. **Flowable 전환**: Section 5.2 의 활성화 방법 참고. 작업량 약 2 day.

### Phase F — 대시보드 통합 (½ day)

20. `ApprovalMapper.countPendingForUser` 를 `approval/countPending` DataSet 메서드로 노출.
21. `PageDashboard.vue` 의 "미결 결재" 위젯을 실 API 로 교체 (현재는 mock 데이터일 수 있음).

22. SSE 수신 시 해당 위젯 실시간 갱신.

---

## 8. 빠른 참조 — DataSet 호출 예시

### 결재함 조회 (PENDING)

```
curl -X POST http://localhost:19090/api/dataset/search ₩
  -H "Authorization: Bearer <jwt>" ₩
  -H "Content-Type: application/json" ₩
  -d '{
    "serviceName": "approval/searchInbox",
    "parameters": {
      "ds_search": { "rows": [{ "boxType": "PENDING", "userNo": "E0032" }] }
    }
  }'
```

### 문서 상세

```
curl -X POST http://localhost:19090/api/dataset/search ₩
  -H "Authorization: Bearer <jwt>" ₩
  -d '{
    "serviceName": "approval/searchDetail",
    "parameters": { "ds_search": { "rows": [{ "docId": 12 }] } }
  }'
```

### 상신

```
curl -X POST http://localhost:19090/api/dataset/search ₩
  -H "Authorization: Bearer <jwt>" ₩
  -d '{
    "serviceName": "approval/submitDocument",
    "parameters": {
      "ds_doc": {
        "rows": [{
          "docTitle": "2026-04 출장 신청",
          "formCode": "BIZTRIP",
          "drafterNo": "E0032",
          "drafterName": "홍길동",
          "drafterDept": "영업1팀",
          "amount": 500000,
          "content": "<p>서울 → 부산 출장</p>"
        }]
      }
    }
  }'
```

### 승인

```
curl -X POST http://localhost:19090/api/dataset/search ₩
  -H "Authorization: Bearer <jwt>" ₩
  -d '{
    "serviceName": "approval/approve",
    "parameters": {
      "ds_search": { "rows": [{ "lineId": 42, "docId": 12, "comment": "확인 완료" }] }
    }
  }'
```

**반려**

```
curl -X POST http://localhost:19090/api/dataset/search ₩
  -H "Authorization: Bearer <jwt>" ₩
  -d '{
    "serviceName": "approval/reject",
    "parameters": {
      "ds_search": { "rows": [{ "lineId": 42, "docId": 12, "comment": "금액 재검토 필요" }] }
    }
  }'
```

---

## 9. 파일 인덱스 (개발 작업 위치)

| 관심사 | 파일 경로 | 역할 |
|---|---|---|
| **Service 메서드** | backend-core/src/main/java/com/platform/v3/core/approval/ApprovalService.java | 공개 API (6 메서드 |
| **Mapper 인터페이스** | backend-core/src/main/java/com/platform/v3/core/approval/mapper/ApprovalMapper.java | 10 메서드 |
| **Mapper XML** | backend-core/src/main/resources/mapper/approval/ApprovalMapper.xml | 143 lines SQL |
| **Flowable Delegate** | backend-core/src/main/java/com/platform/v3/core/approval/flowable/*.java | 4 클래스 (현재 미 |
| **BPMN 프로세스** | backend-core/src/main/resources/processes/*.bpmn20.xml | 4 파일 (순차/병렬/ |
| **DB 스키마** | backend-core/src/main/resources/db/migration/V{1..7}__*.sql | Flyway 마이그레이 (ap_document/ap_ 은 실제 마이그레이 런타임 상태에만 존 신규 작성 권장) |
| **UI 페이지** | ui/src/pages/PageApproval.vue | 76 lines, 결재함 + |
| **Keycloak JWT 검** | backend-core/src/main/java/com/platform/v3/core/config/SecurityConfig.java | Bearer token → cu |

| 증 | | |
|---|---|---|
| **DataSet 프로토콜** | `backend-core/src/main/java/com/platform/v3/core/dataset/` | DataSetController, ServiceRegistry, @DataSetServiceM |

---

## 10. 주의 사항

- **권한 체크 부재**: 현재 `ApprovalService` 는 `currentUser` 를 받지만 실제로 "이 사용자가 이 docId 에 접근할 권한이 있는가" 를 검증하지 않는다. 개발 시 `ds_doc.drafter_no == currentUser` 또는 `ds_line.approver_no == currentUser` 검증 로직 추가 필수.
- **트랜잭션**: submit/approve/reject 는 `@Transactional` 적용됨. select 계열은 트랜잭션 없음 (정상).
- **NotificationService SSE**: 알림 수신자가 `approverId = 10L` 하드코딩. 실제 운영 전에 `ap_approval_line.approver_no` → `org_employee.employee_id` 변환 후 전달 필수.
- `PageApproval.vue` **의** `userId: 1` **파라미터**: MyBatis 는 `userNo` 를 기대하여 매핑되지 않고 default `E0032` 가 적용됨. 결과적으로 **로그인 사용자와 무관한 고정 사용자 기준 리스트**가 조회되는 버그. Phase A 에서 반드시 수정.

---

**문서 끝 — 개발 시 본 문서와 CLAUDE.md, warn.md, TODO.md 를 병행 참고.**